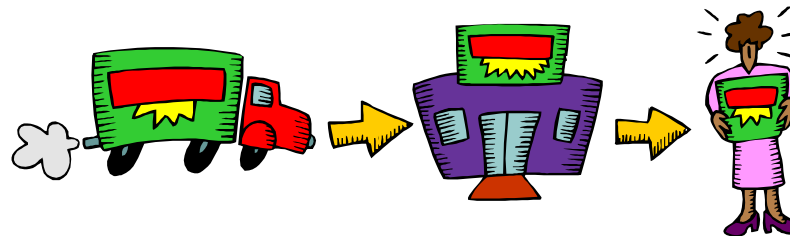


Công nghệ phần mềm

Một số phương pháp kiểm thử cơ bản



Nội dung

- Kiểm thử dựa trên phân tích giá trị biên
- Kiểm thử giá trị đặc biệt
- Kiểm thử giá trị ngẫu nhiên
- Kiểm thử dựa trên phân hoạch tương đương
- Kiểm thử dựa trên bảng quyết định

KIỂM THỬ DỰA TRÊN PHÂN TÍCH GIÁ TRỊ BIÊN

Giới thiệu

- Các chương trình có thể coi là một hàm P
 - Input hàm: MIỀN XÁC ĐỊNH của hàm (là các khoảng hợp lệ của biến)
 - Output hàm
- Ví dụ: `gcd(int x1, int x2)` có miền xác định
$$-2^{31} \leq x1, x2 \leq 2^{31} - 1$$
- Giả định: Lỗi có thể xảy ra ở BIÊN hoặc GẦN BIÊN
- Bốn loại kiểm thử (KT) biên
 - KT giá trị biên (bình thường) - BVA
 - KT giá trị biên mạnh
 - KT giá trị trường hợp xấu nhất
 - KT giá trị đặc biệt

Loại 1 – KT giá trị biên bình thường

- Giả thiết khiếm khuyết đơn: “Hỏng hóc xảy ra hiếm khi do hai (hoặc hơn) khiếm khuyết cùng xảy ra”
- Là kỹ thuật kiểm thử hàm phổ biến nhất
- Sử dụng miền xác định để xây dựng ca kiểm thử

Ca kiểm thử

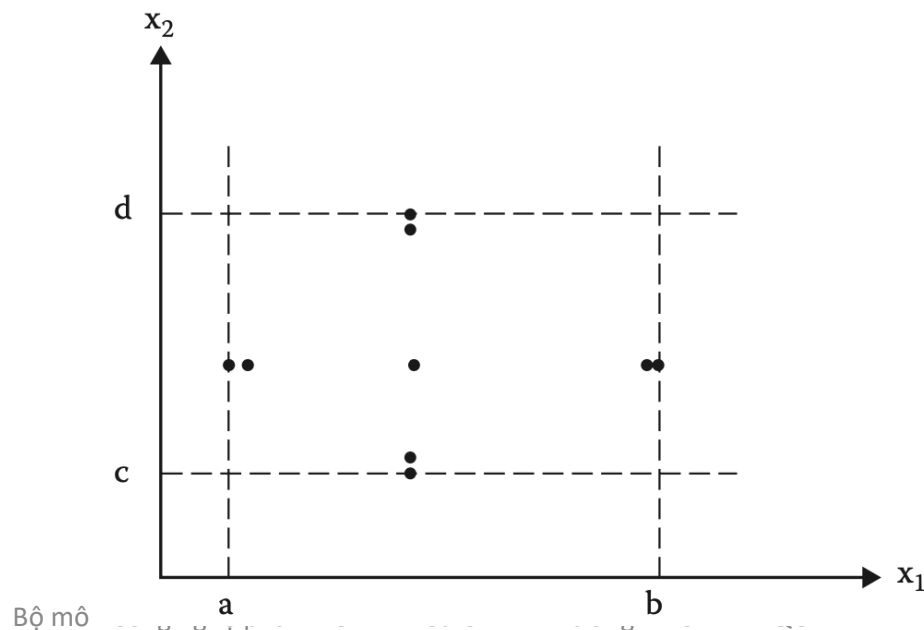
- Tư tưởng
 - Một biến sẽ nhận các giá trị sau
 $\text{min}, \text{min} + 1, \text{nom}, \text{max} - 1, \text{max}$
(nom thường là giá trị $(\text{min} + \text{max}) / 2$)
 - Cho $n - 1$ biến nhận giá trị norm, biến còn lại nhận 5 giá trị
 - Cần $4n + 1$ ca kiểm thử cho hàm n biến

Ví dụ

- `bool isPrime(int a)`
 - Trả về True nếu a là số nguyên tố, ngược lại trả về False
- `bool isLeapYear(int year)`
 - Trả về True nếu year là năm nhuận, ngược lại trả về False
- `int gcd(int x1, int x2)`
- `int isTriangle(int a, b, c)`
 - Trả về -1 nếu không phải tam giác
 - Nếu là tam giác đều, vuông, cân, trả về các giá trị khác -1
- `Date getNextday(int month, int day, int year)`
 - Trả về ngày kế tiếp

Ví dụ - GCD

- Viết các ca kiểm thử biên cho hàm $\text{gcd}(\text{int } x_1, \text{int } x_2)$
 - $S(x_1) = S(x_2) = \{-2^{31} (\text{min}), -2^{31} + 1, 0 (\text{norm}), 2^{31} - 2, 2^{31} - 1\}$
 - Cho x_1 nhận giá trị norm (= 0), giá trị x_2 thay đổi thuộc S
 - Cho x_2 nhận giá trị norm, giá trị x_1 thay đổi thuộc S
 - Tổng: 9 ca kiểm thử



Ví dụ - Kiểm tra tam giác

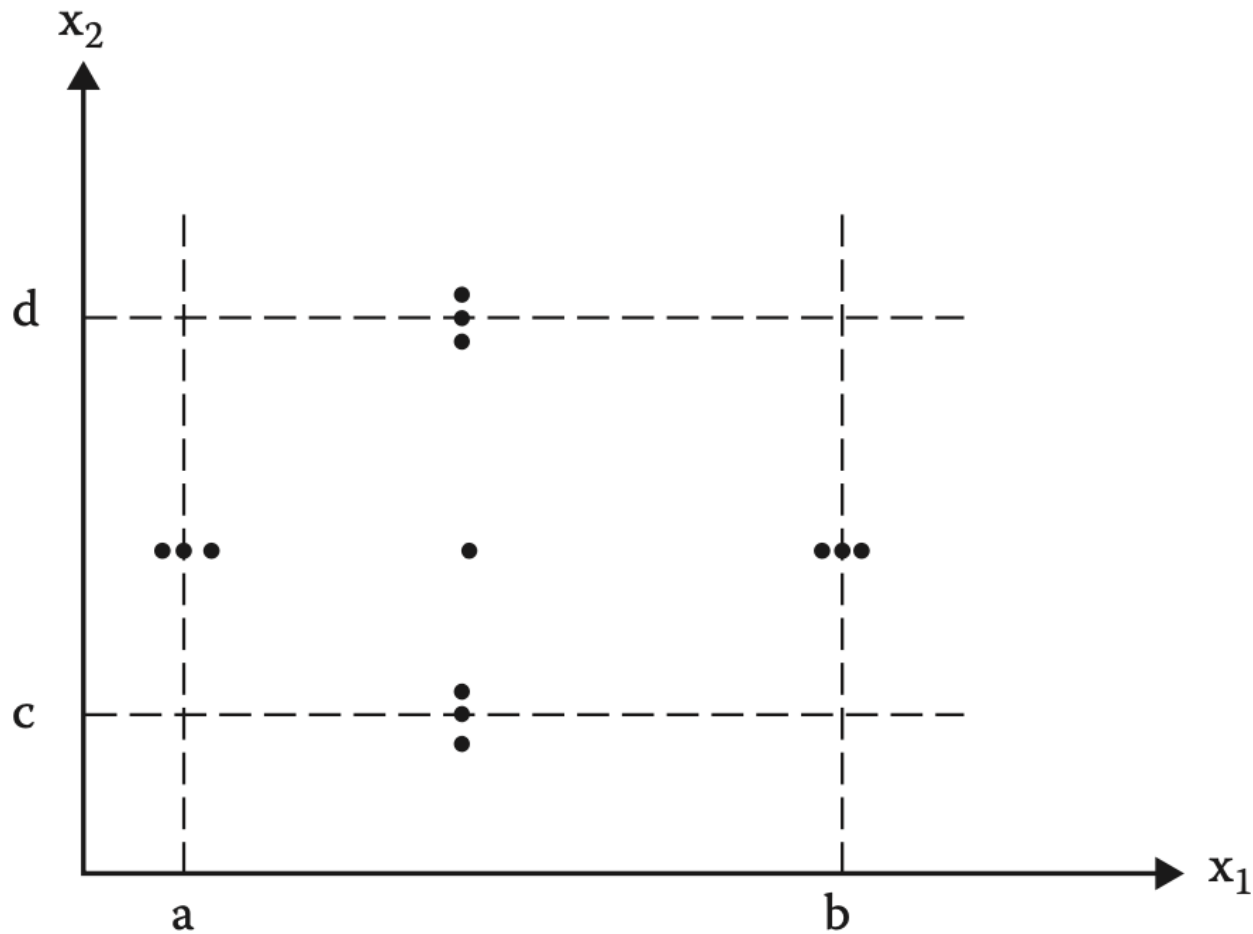
- Biên dưới = 1
- Biên trên là $2^{31} - 1$, tuy nhiên, ở đây biên trên được quy định trong đặc tả bằng 200
- Một biến có thể nhận giá trị: 1, 2, 100, 199, 200
- Ba biến => 13 ca kiểm thử (bỏ đi hai ca kiểm thử lặp)

Case	a	b	c	Expected Output
1	100	100	1	Isosceles
2	100	100	2	Isosceles
3	100	100	100	Equilateral
4	100	100	199	Isosceles
5	100	100	200	Not a triangle
6	100	1	100	Isosceles
7	100	2	100	Isosceles
8	100	100	100	Equilateral
9	100	199	100	Isosceles
10	100	200	100	Not a triangle
11	1	100	100	Isosceles
12	2	100	100	Isosceles
13	100	100	100	Equilateral
14	199	100	100	Isosceles
15	200	100	100	Not a triangle

Loại 2 – KT biên mạnh

- Kiểm thử biên mạnh (robustness testing) là một mở rộng đơn giản của BVA
- Ngoài năm giá trị biên bổ sung thêm hai giá trị ngoài biên:
 - Giá trị ngay trên giá trị cực đại (max+) và
 - Giá trị ngay dưới giá trị cực tiểu (min-).
- Mục đích chính là xem chương trình có kiểm tra giá trị hợp lệ của đầu vào không.
- Số ca kiểm thử: $6n + 1$

Ví dụ - GCD



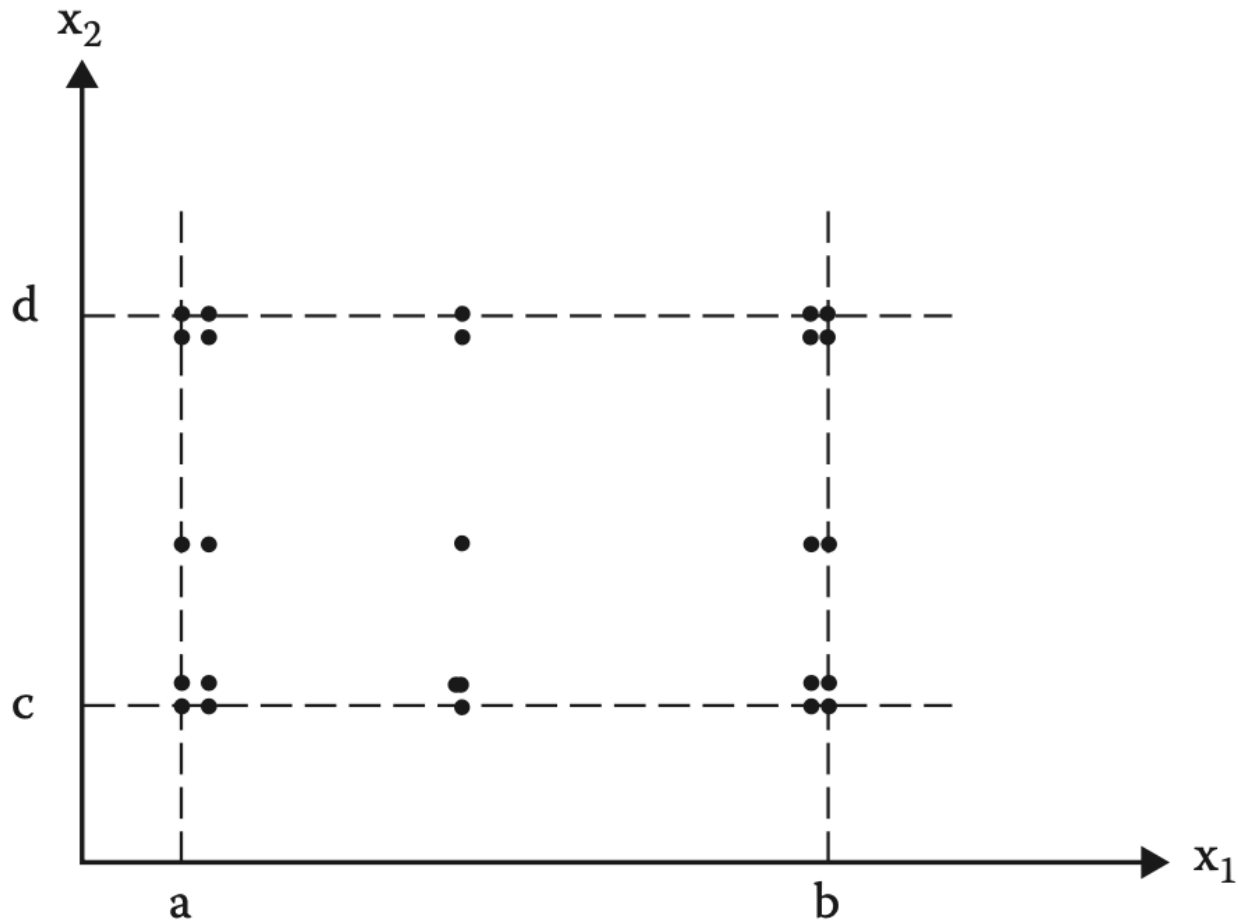
Hạn chế

- BVA (mạnh) hiệu quả với các chương trình có các đầu vào độc lập nhau, không quan tâm ràng buộc của các biến
- Ví dụ
 - Hàm getNextday(int month, int day, int year) có điều kiện giữa các biến khá phức tạp (BVA bỏ qua ràng buộc các biến để sinh ra kiểm thử)

Loại 3 - KT trường hợp xấu nhất

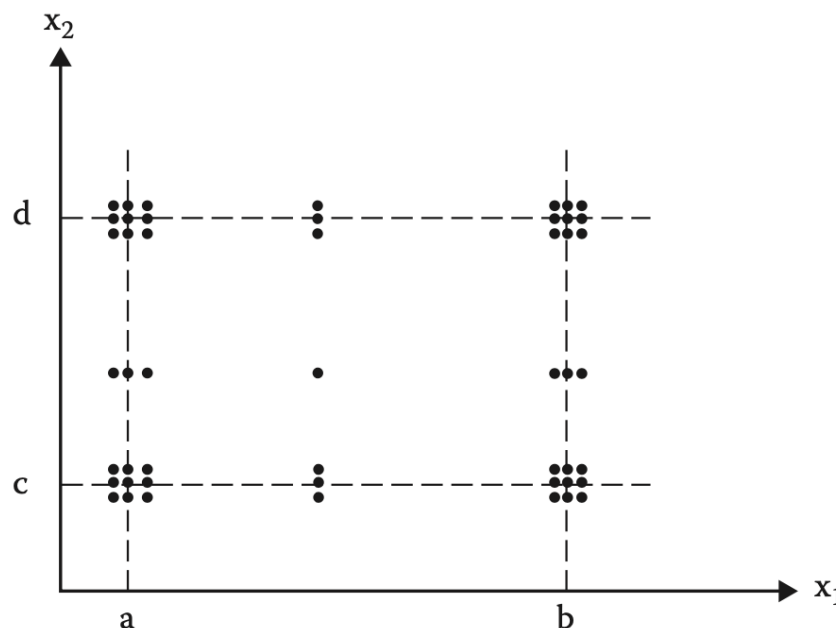
- Khi **các biến có tương tác với nhau** thì cần kiểm tra các bộ giá trị kết hợp các cực trị này
 - Điều gì xảy ra khi nhiều hơn một biến nhận các giá trị (gần) cực trị?
- Nếu hàm có n biến
 - Một biến nhận giá trị $\{\min, \min + 1, \text{norm}, \max - 1, \max\}$
 - Số ca kiểm thử $= 5^n$
- Khi nào nên dừng?
 - Khi các biến trong hàm có ảnh hưởng lớn đến nhau
 - Hàm hoạt động sai sẽ gây ra hậu quả to lớn

Ví dụ - GCD



Loại 4 - KT trường hợp xấu nhất mạnh

- Nếu hàm có n biến
 - Một biến nhận giá trị $\{\text{MIN} - 1, \text{min}, \text{min} + 1, \text{norm}, \text{max} - 1, \text{max}, \text{MAX} + 1\}$
 - Số ca kiểm thử = 7^n



Suy nghĩ về những loại biến khác

- Ta mới tìm hiểu các hàm có tham số kiểu số
- Trong thực tế có nhiều kiểu dữ liệu khác nhau
 - Xâu, ký tự
 - Mảng (một chiều, nhiều chiều)
 - Con trỏ (một mức, nhiều mức)
 - Kiểu cấu trúc
 - v.v.
- Hãy thử nghĩ cách kiểm thử cho những hàm dùng kiểu biến này?

KIỂM THỬ GIÁ TRỊ ĐẶC BIỆT

Kiểm thử giá trị đặc biệt

- Kiểm thử giá trị đặc biệt là phương pháp được thực hiện nhiều nhất trên thực tế, nó cũng trực quan nhất, và không có dạng cố định nhất
- Sử dụng kỹ nghệ và kiến thức miền ứng dụng để phán đoán và đưa ra ca kiểm thử
- Mặc dù mang tính chủ quan cao, đây vẫn là phương pháp hiệu quả để phát hiện khiếm khuyết của chương trình

KIỂM THỬ GIÁ TRỊ NGẪU NHIÊN

Kiểm thử giá trị ngẫu nhiên

- Thử các giá trị ngẫu nhiên (fuzzy testing)
- Chạy nhiều giờ, nhiều ngày, nhiều tuần, v.v.
- Ví dụ:
 - Windows NT (2000): 21% ứng dụng bị crash
 - MacOSX (2006): kiểm thử 135 tiện ích, crash 7%
 - Linux, FreeBSD, MacOS (2020)
 - Kiểm thử 75 tiện ích trên từng nền tảng
 - Linux: 12% lỗi, MacOS 16%, FreeBSD 19%

Ví dụ – isTriangle(int a, int b, int c)

- Viết bao nhiêu ca kiểm thử là đủ?

<i>Test Cases</i>	<i>Nontriangles</i>	<i>Scalene</i>	<i>Isosceles</i>	<i>Equilateral</i>
1289	663	593	32	1
15,436	7696	7372	367	1
17,091	8556	8164	367	1
2603	1284	1252	66	1
6475	3197	3122	155	1
5978	2998	2850	129	1
9008	4447	4353	207	1
Percentage	49.83%	47.87%	2.29%	0.01%

KIỂM THỬ LỚP TƯƠNG ĐƯƠNG (ECT)

Kiểm thử lớp tương đương

- Lý do:
 - Cảm giác kiểm thử hết
 - Tránh dư thừa
- Các lớp tương đương tạo thành một phân hoạch của miền dữ liệu
 - Hợp của tất cả các lớp bằng miền đầu vào
 - Cảm giác đã kiểm thử hết
 - Hai lớp bất kỳ không giao nhau
 - Không dư thừa

Các lớp tương đương

- Ý tưởng của ECT là chỉ kiểm thử với một phần tử của mỗi miền tương đương
 - Giảm rất nhiều dư thừa tiềm tàng nếu các lớp tương đương được chọn hợp lý
- Mấu chốt là làm sao chọn được quan hệ tương đương để từ đó xác định được các lớp tương đương (phân hoạch)

Chọn phân hoạch

- Thường là “thủ công” (craft):
 - Không dựa trên mã nguồn, chỉ dựa trên đặc tả
 - Cần hiểu biết về miền xác định, thường không thể xác định dựa vào đặc tả thiết kế giao diện
 - Phải hiểu đầu vào phụ thuộc nhau như thế nào

Ví dụ

- Xét chương trình P có ba biến đầu vào: a, b và c với các miền xác định là A, B, and C.
- Phân hoạch của các miền này giả sử là:

$$A = A_1 \cup A_2 \cup A_3$$

$$B = B_1 \cup B_2 \cup B_3 \cup B_4$$

$$C = C_1 \cup C_2$$

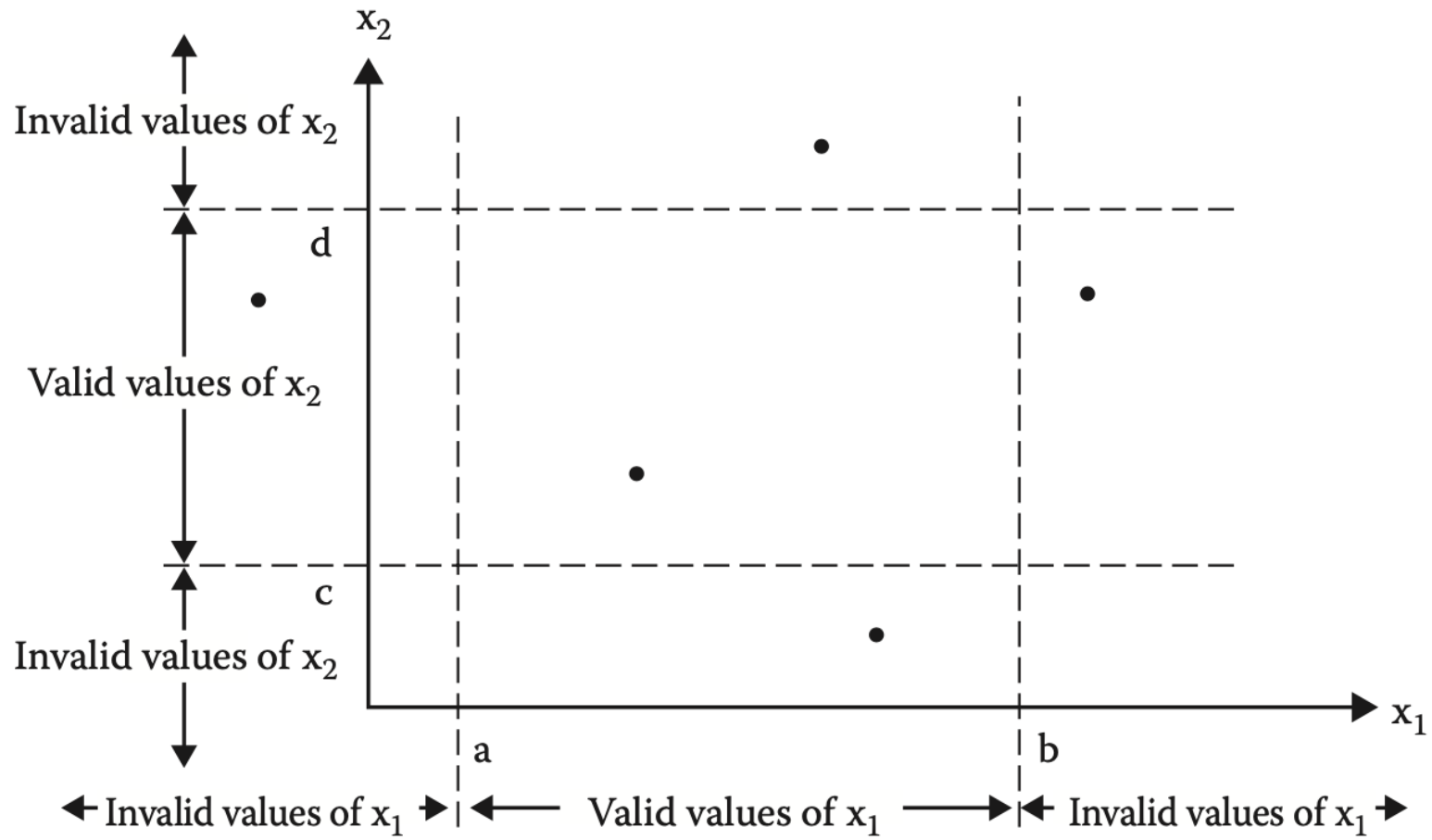
Ví dụ (tiếp)

- Gọi a_i thuộc A_i là một phần tử đại diện của lớp
 - Ví dụ lấy phần tử giữa của 1 khoảng
- Tương tự có b_i và c_i .
- Các ca kiểm thử sẽ được xây dựng từ các phần tử đại diện này
- Ý tưởng ở đây là phần tử đại diện này cũng tốt như các phần tử khác ở trong cùng lớp tương đương đó

Loại 1 - ECT truyền thống

- Chỉ phân biệt lớp giá trị hợp lệ và không hợp lệ
- Tư tưởng chọn ca kiểm thử giống BVA
- Ca kiểm thử
 - Tc0: mọi biến nhận giá trị hợp lệ
 - Tc1: cố định $n-1$ biến, thay đổi giá trị biến còn lại

ECT truyền thống - $\text{GCD}(x_1, x_2)$



Loại 2 - ECT yếu

- ECT yếu chỉ lấy tất cả các phần tử đại diện ít nhất một lần
- Số ca kiểm thử tối thiểu sẽ bằng số lớp của phân hoạch có nhiều tập con nhất
 - Trong ví dụ trước là 4

#	a	b	c
WE1	a_1	b_1	c_1
WE2	a_2	b_2	c_2
WE3	a_3	b_3	c_1
WE4	a_1	b_4	c_2

Loại 3 - ECT mạnh

- ECT mạnh dựa trên tích Đề-các của các lớp con
- Với ví dụ trước ta có:

$$3 * 4 * 2 = 24 \text{ ca kiểm thử}$$

- Cách này xét đến tất cả các tương tác của các giá trị đại diện

#	a	b	c
SE1	a1	b1	c1
SE2	a1	b1	c2
SE3	a1	b2	c1
SE4	a1	b2	c1
SE5	a1	b3	c1
SE6	a1	b3	c2
SE7	a1	b4	c1
SE8	a1	b4	c2
SE9	a2	b1	c1
SE10	a2	b1	c2
SE11	a2	b2	c1
SE12	a2	b2	c2
SE13	a2	b3	c1
SE14	a2	b3	c2
SE15	a2	b4	c1
SE16	a2	b4	c2
SE17	a3	b1	c1
SE18	a3	b1	c2
SE19	a3	b2	c1
SE20	a3	b2	c2
SE21	a3	b3	c1
SE22	a3	b3	c2
SE23	a3	b4	c1
SE24	a3	b4	c2

BẢNG QUYẾT ĐỊNH

Bảng quyết định

- Yêu cầu chức năng có thể mô tả bằng bảng quyết định (DT)
- DT là một cách chính xác và gọn để mô tả logic phức tạp
 - Gắn các điều kiện với các hành động tương ứng
 - Giống lệnh if-then-else và switch-case
- DT có thể liên kết nhiều điều kiện độc lập với vài hành động một cách dễ hiểu
 - Khác các cấu trúc điều khiển trong các ngôn ngữ lập trình

Ví dụ về bảng quyết định

Điều kiện	Máy in không in	Y	Y	Y	Y	N	N	N	N
	Đèn đỏ nhấp nháy	Y	Y	N	N	Y	Y	N	N
	Không nhận ra máy in	Y	N	Y	N	Y	N	Y	N
Hành động	Kiểm tra dây nguồn			X					
	Kiểm tra dây tín hiệu	X		X					
	Kiểm tra phần mềm in đã cài đúng	X		X		X		X	
	Kiểm tra/thay mực	X	X			X	X		
	Kiểm tra kẹt giấy		X		X				

Sử dụng bảng quyết định

- Để quan sát tất cả các điều kiện dễ dàng
- Có thể dùng để
 - Mô tả logic phức tạp
 - Sinh ca kiểm thử, còn gọi là *kiểm thử dựa trên logic*
- Kiểm thử dựa trên logic được xem là:
 - Kiểm thử cấu trúc khi áp dụng cho các cấu trúc chương trình
 - Vd luồng điều khiển
 - Kiểm thử hàm khi áp dụng cho đặc tả.

Cấu trúc bảng quyết định

1. Các điều kiện	2. Các giá trị điều kiện
3. Hành động	4. Xảy ra hay không

1. Mỗi điều kiện tương ứng với một biến, một quan hệ, hoặc một mệnh đề (predicate – ví dụ ‘n là số nguyên tố’)
2. Các giá trị của điều kiện
3. Mỗi hành động là một thủ tục hoặc thao tác phải thực hiện
4. Đánh dấu hành động có/không xảy ra

Ví dụ - bài toán tam giác

c1: $a < b + c$?	F	T	T	T	T	T	T	T	T	T	T
c2: $b < a + c$?	—	F	T	T	T	T	T	T	T	T	T
c3: $c < a + b$?	—	—	F	T	T	T	T	T	T	T	T
c4: $a = b$?	—	—	—	T	T	T	T	F	F	F	F
c5: $a = c$?	—	—	—	T	T	F	F	T	T	F	F
c6: $b = c$?	—	—	—	T	F	T	F	T	F	T	F
a1: Not a triangle	X	X	X								
a2: Scalene											X
a3: Isosceles							X		X	X	
a4: Equilateral				X							
a5: Impossible					X	X		X			

Ví dụ - bài toán tam giác

<i>Case ID</i>	a	b	c	<i>Expected Output</i>
DT1	4	1	2	Not a triangle
DT2	1	4	2	Not a triangle
DT3	1	2	4	Not a triangle
DT4	5	5	5	Equilateral
DT5	?	?	?	Impossible
DT6	?	?	?	Impossible
DT7	2	2	3	Isosceles
DT8	?	?	?	Impossible
DT9	2	3	2	Isosceles
DT10	3	2	2	Isosceles
DT11	3	4	5	Scalene

Phương pháp xây dựng bảng

1. Xác định các điều kiện và giá trị của chúng
2. Xác định số luật tối đa
3. Xác định các hành động
4. Đánh số các luật nếu cần
5. Đánh số các hành động thích hợp cho mỗi luật
6. Kiểm tra chính sách
7. Đơn giản hóa các luật (gộp cột)

Sử dụng bảng quyết định

- Bảng thích hợp khi:
 - Đặc tả có thể chuyển về dạng bảng
 - Thứ tự các hành động xảy ra không quan trọng
 - Thứ tự các luật không ảnh hưởng đến hành động
 - Khi một luật thỏa mãn và được chọn thì không cần xét luật khác
- Các hạn chế trên không ảnh hưởng đến việc sử dụng bảng
 - Trong hầu hết các ứng dụng thứ tự các mệnh đề được xét là không quan trọng

Một số vấn đề với bảng quyết định

- Trước khi sử dụng bảng cần đảm bảo:
 - Các luật phải đầy đủ
 - Có mọi tổ hợp
 - Các luật phải nhất quán
 - Mọi tổ hợp giá trị chân lý chỉ gây ra một hoặc một tập hành động

Thiết kế ca kiểm thử

- Khi đặc tả đã được kiểm tra, mục tiêu là chứng tỏ chương trình thực hiện các hành động đúng cho mọi tổ hợp các giá trị của mệnh đề
 - Nếu có k luật và n mệnh đề đúng/sai, thì có ít nhất k trường hợp và nhiều nhất là 2^n trường hợp phải xét.
- Có thể dựa trên các luật chưa mở rộng hoặc các luật đã mở rộng với 2^n ca
 - Xác định đầu vào cho mỗi ca

Thiết kế ca kiểm thử

- Để xác định ca kiểm thử, chúng ta chuyển các điều kiện thành đầu vào, hành động thành đầu ra.
- Một số điều kiện sẽ tham chiếu đến các lớp tương đương đầu vào, và hành động tham chiếu đến các phần xử lý chức năng chính của cột đang xét.
- Các luật được chuyển thành các ca kiểm thử

Ví dụ – getNextDate

M1 = {month: month has 30 days}

M2 = {month: month has 31 days}

M3 = {month: month is February}

D1 = {day: $1 \leq \text{day} \leq 28$ }

D2 = {day: day = 29}

D3 = {day: day = 30}

D4 = {day: day = 31}

Y1 = {year: year is a leap year}

Y2 = {year: year is not a leap year}

Ví dụ – getNextDate

<i>Conditions</i>											
c1: Month in M1?	T										
c2: Month in M2?		T									
c3: Month in M3?			T								
c4: Day in D1?											
c5: Day in D2?											
c6: Day in D3?											
c7: Day in D4?											
c8: Year in Y1?											
a1: Impossible											
a2: Next date											

Tham khảo

- Software Testing: A Craftsman's Approach, Fourth Edition

Tổng kết

- Kiểm thử dựa trên phân tích giá trị biên
- Kiểm thử dựa trên phân hoạch tương đương
- Kiểm thử dựa trên bảng quyết định
- V.v